Probabilistic Roadmaps (PRM)

Contents

1. Overview . . . 2
2. Building a roadmap from random samples . . . 4
   - 2.1 Why sample instead of building the free space . . . 4
   - 2.2 Phase 1 and phase 2: milestones and edges . . . 4
   - 2.3 The query: joining a trip to the roadmap . . . 6
3. Many trips on one roadmap: single-query and multi-query planners . . . 7
4. What makes a graph a roadmap: accessibility and connectivity . . . 8
5. Can a planner miss a route? Completeness . . . 10
   - 5.1 Complete and resolution complete . . . 10
   - 5.2 Probabilistically complete . . . 11
   - 5.3 Why more milestones help: the chance of missing . . . 12
6. Narrow passages and smarter sampling . . . 14
   - 6.1 The tray robot: where uniform samples starve . . . 15
   - 6.2 Gaussian sampling and the bridge test . . . 16
7. The visibility roadmap: guards and connectors . . . 19
8. Lazy collision checking . . . 21
9. Summary . . . 23
10. Sources . . . 24
11. Key terms . . . 25

Where this fits:

- **Builds on:** Normal distribution; Uniform distribution; Graphs and state spaces; A* search; Basic motion planning problem; Checking a path segment; Collision detection.
- **Leads to:** Manipulation planning (transit and transfer); MoveIt and OMPL; Preimage planning.
- **Compare with:** Rapidly-exploring random tree (RRT).

1. Overview

Key point: A probabilistic roadmap scatters random free configurations over C-space, joins nearby ones by checked straight edges, and searches the resulting graph. Built once, it answers every trip in an unchanged room for a few hundred checks each; the price is that a missed route is only unlikely, not impossible, and tight gaps need smarter sampling.

So far the robot can plan in its room in two ways. A* (G-2577) on the 0.5 m grid finds a 3.414 m route to the door. The rapidly-exploring random tree (G-2761), RRT for short, grows a tree of random configurations from the start until a branch reaches the goal, so it needs no grid and never builds the obstacle region (G-2650).

What still breaks is a day of work. The robot, a disk of radius 0.25 m whose grown obstacles (G-2652) mark where its centre may not go, makes four trips in a room that does not change (Figure 1), each starting where the last one ended:

1. its dock at (2, 1) to the doorway centre (3.6, 4.0);
2. the door to a spot beside the cabinet, (5.9, 2.0);
3. that cabinet spot to the corner behind the green pole, (6.6, 0.4);
4. the corner back to the dock.

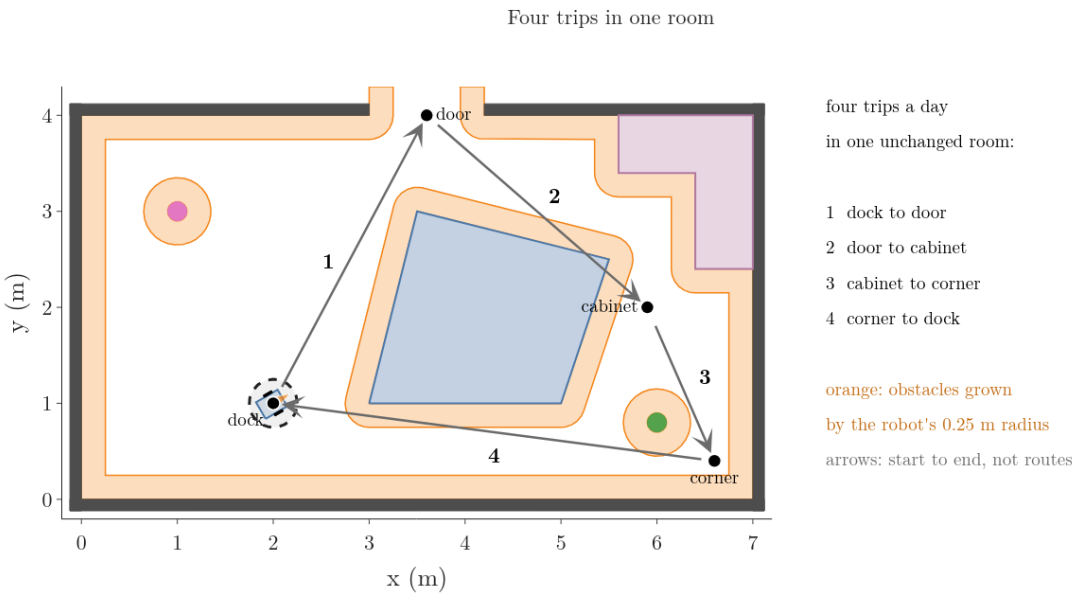


Figure 1: The room in C-space: the obstacles grown by the robot's 0.25 m radius (orange), the robot at its dock (2, 1), and four numbered grey arrows from the start to the end of each trip of the day: 1 dock to door (3.6, 4.0), 2 door to cabinet (5.9, 2.0), 3 cabinet to corner (6.6, 0.4), 4 corner to dock. The arrows join start and end; they are not routes

A tree grown from the dock serves trip 1 only. Trip 2 starts at the door, so the tree is thrown away and the sampling and collision checks start again from nothing, four times a day, in a room that has not changed. A second problem waits when the robot carries a round tray 0.8 m across: then some gaps leave its centre only 0.1 m to 0.2 m of room, and random samples rarely land in them.

The opening question of this Note: **can the robot build one graph of its free space from random samples, answer all four trips from it, and still get through tight gaps?** We answer it in steps:

- building the graph, the probabilistic roadmap, and answering trip 1 (Section 2);
- reusing it for all four trips (Section 3);
- what the graph must do so that no trip fails (Section 4);
- how sure an answer of "no route" is (Section 5);
- tight gaps, and two ways of sampling that find them (Section 6);
- a smaller graph that covers the room just as well (Section 7);
- checking only the edges a route uses (Section 8).

2. Building a roadmap from random samples

> **Key point:** Build a graph of the free space once, in two phases: draw random configurations and keep the free ones as milestones, then join each milestone to its 10 nearest by straight edges that pass a collision check. A trip only joins its two ends to the graph and runs A*.

2.1 Why sample instead of building the free space

The four trips share one room, so the robot should describe the room once and reuse the description. Describing it exactly means building the obstacle region, which is costly for real robots (why planners do not build the obstacle region). A grid avoids that, but its size grows with the number of joints. With 10 values for each joint:

- d is the number of joints, the dimension of the C-space.

$$\text{cells} = 10^d$$

$$d = 2: \quad 10^2 = 100$$

$$d = 3: \quad 10^3 = 1{,}000$$

$$d = 7: \quad 10^7 = 10{,}000{,}000$$

(KAIST PRM slides). Ten million cells for a 7-joint arm, each needing a collision check, is the **curse of dimensionality** (G-520), the work and data that grow with the number of dimensions, here in C-space (MR §10.5).

The fix is the idea a road map uses. A road atlas does not draw every field of the country; it draws a network of roads, and any trip goes from home to the nearest road, along roads, and off at the nearest road to the destination (MR §10.3). We build such a network inside the free space from random configurations, checked one at a time by collision detection (G-2665), so the obstacle region is never built (MR §10.5.2). Like the RRT, the method is a sampling-based planner (G-2760): random configurations and collision checks take the place of a grid (MR §10.5). How many samples it needs depends on how much of the free space one configuration can see along straight lines, and that share does not usually fall with every added dimension the way a grid's cell count rises (MR §10.5.2).

2.2 Phase 1 and phase 2: milestones and edges

The network has two parts, one per phase (MR §10.5.2, Algorithm 10.4):

1. **Phase 1, sample:** draw configurations from the uniform distribution (G-467) over the room's floor, every place equally likely, and check each; keep the free ones, drop the rest, until 100 are kept.
2. **Phase 2, connect:** for each kept configuration, find its 10 nearest neighbours (G-1306) among the kept ones and try the straight segment to each; add the segment if edge resolution (G-2679) finds it free, checking configurations at most 0.05 m apart.

A free configuration kept as a vertex is a **milestone** (G-2781) (Hauser ch.10; KAIST PRM slides). The method that tries to join two configurations, here the straight segment checked by edge resolution, is the **local planner** (G-2782): a fast planner that proposes one simple motion, without searching round obstacles, and reports whether that motion is free (MR §10.5.2; Kavraki et al. 1996). The RRT's steer (G-2763) stops after at most 0.5 m; the local planner must reach the other milestone exactly, since an edge joins two milestones (MR §10.5.2). The milestones and the free segments between them form a **graph** (G-2540), the milestones its vertices and the segments its edges; this graph is the **probabilistic roadmap** (G-2780), PRM for short, a method published in 1996 (Kavraki et al. 1996).

Figure 2 builds one, seed 0. Watch phase 1: blue dots are kept milestones and red crosses are draws that landed in the grown obstacles. Then watch phase 2, where blue lines are added edges and dotted red lines are edges that cut a grown obstacle and are dropped.

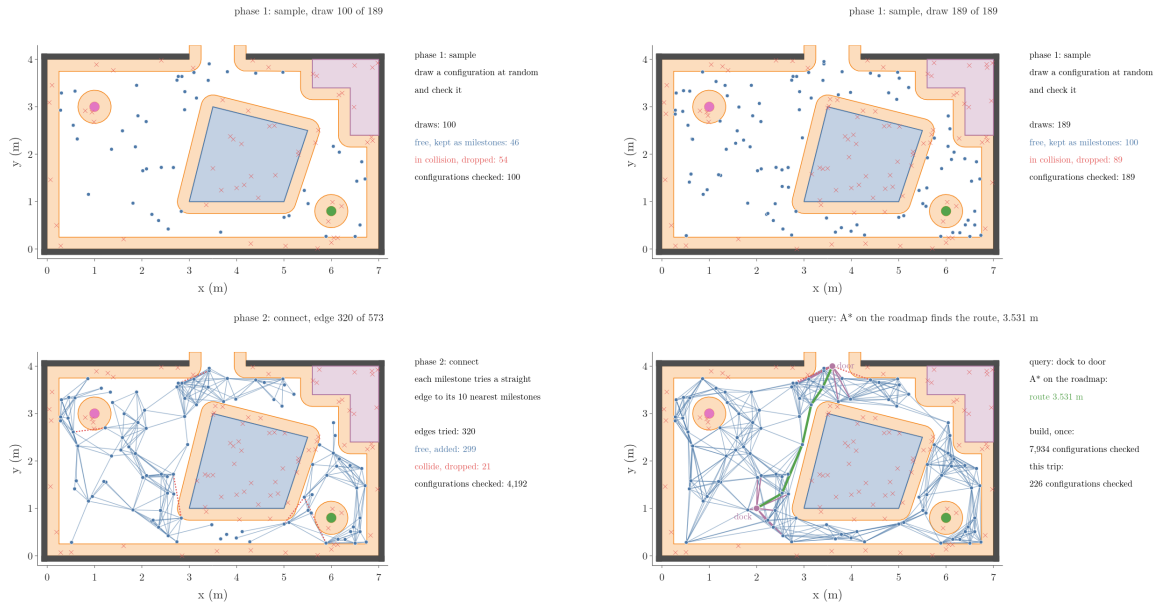


Figure 2: Animation: building a probabilistic roadmap in the room's C-space (obstacles grown by 0.25 m, orange). Phase 1: draws appear ten at a time; 100 free draws are kept as milestones (blue) and 89 in collision are dropped (red crosses), 189 configurations checked. Phase 2: each milestone tries straight edges to its 10 nearest milestones; 573 edges tried, 525 free (blue lines), 48 cut an obstacle and are dropped (dotted red); 7,745 more configurations checked along the edges. Query: the dock (2, 1) and the door (3.6, 4.0) each try their 10 nearest milestones (purple, 18 added), and A* finds the route (green), 3.531 m, for 226 checks. Last frame: build 7,934 checks once, this trip 226

The counts of phase 1. The robot's centre is free on 15.02 m^2 of the 28 m^2 floor (notebook):

$$15.02/28 = 0.536$$

so about half the draws are free: 100 milestones took 189 draws, and 89 landed in the grown obstacles.

The counts of phase 2. Each of the 100 milestones tries 10 neighbours:

- N is the number of milestones, 100;
- k is the number of neighbours each tries, 10.

$$kN = 10 \times 100 = 1{,}000 \text{ tries}$$

A pair in which each milestone picks the other is tried only once. If every pair were picked both ways, the count would halve:

$$kN/2 = 1{,}000/2 = 500$$

So the distinct edges tried lie between 500 and 1,000; our room gave 573, of which 525 were free (notebook). Checking them cost 7,745 configuration checks, against 189 for phase 1: the edges, not the milestones, are where a roadmap spends its effort (Hauser ch.10, "Lazy collision checking in PRMs").

The pseudocode (MR §10.5.2, Algorithm 10.4), with R the roadmap:

1. for each of N times: draw a free configuration q and add it to R;
2. for each milestone q: for each q' among the k nearest milestones of q: if the local planner finds the segment from q to q' free and the edge is not in R yet, add it to R.

The roadmap is built, but it does not yet contain the dock or the door.

2.3 The query: joining a trip to the roadmap

To answer trip 1, the dock and the door must join the roadmap. Each tries the straight segment to its 10 nearest milestones, like a new milestone; then A* (G-2577), with the straight-line distance as edge cost and as heuristic (G-2570), searches the joined graph (MR §10.5.2).

In the last two frames of Figure 2, the purple segments join the dock and the door: 20 tried, 18 free, 226 configuration checks. A* returns the green route, 3.531 m, through 4 milestones. The straight drive to the door is 3.4 m; the roadmap's route zigzags between milestones and is 0.131 m longer:

$$3.531 - 3.4 = 0.131 \text{ m}$$

Building the roadmap and answering a trip are separate steps, and they cost very different amounts: 7,934 checks to build, 226 for trip 1 (notebook). The two steps are the **learning phase** (G-2784), which builds the roadmap, and the **query phase** (G-2785), which joins a start and a goal to it and searches (Kavraki et al. 1996; LaValle §5.6.1 calls the first the preprocessing phase). Trip 1 is answered; the next section asks what the other three trips cost.

3. Many trips on one roadmap: single-query and multi-query planners

> **Key point:** A roadmap built once answers every trip in an unchanged room: the four trips cost 7,934 checks for the build plus 842 for the trips, against 32,578 if a new roadmap were built for each. A planner that builds such a reusable graph is a multi-query planner; one that solves each trip from nothing, like the RRT, is a single-query planner.

The opening problem was trips 2 to 4: a tree grown for trip 1 serves no other trip. The roadmap of Section 2, its milestones (the kept random configurations) and checked edges, holds no start or goal, so any trip can use it. Figure 3 answers the four trips on the one roadmap; for each trip, watch the two ends join (purple), the route appear (green), and the bars on the right add up the checks.

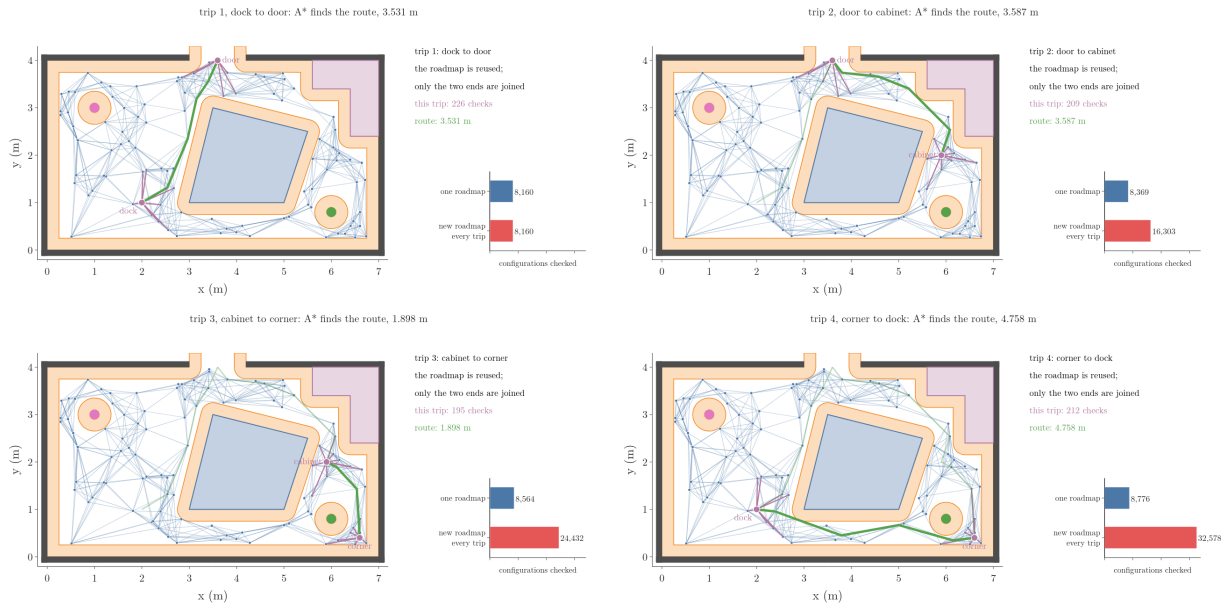


Figure 3: Animation: the four trips answered on the same roadmap of 100 milestones and 525 edges (light blue). For each trip, its two ends try their 10 nearest milestones (purple) and A* finds the route (green): trip 1 dock to door 3.531 m for 226 checks, trip 2 door to cabinet 3.587 m for 209, trip 3 cabinet to corner 1.898 m for 195, trip 4 corner to dock 4.758 m for 212. Earlier routes stay faintly green. Bars: configurations checked so far with one roadmap (blue, 8,160, 8,369, 8,564, 8,776) and with a new roadmap for every trip (red, 8,160, 16,303, 24,432, 32,578)

The four trips cost 226, 209, 195 and 212 checks (notebook). With one roadmap:

$$226 + 209 + 195 + 212 = 842$$

$$7{,}934 + 842 = 8{,}776 \text{ checks}$$

With a new roadmap for each trip:

$$4 \times 7{,}934 + 842 = 32{,}578 \text{ checks}$$

The largest trip costs 2.8 percent of the build:

$$226/7{,}934 = 0.028$$

So after the build, a trip is almost free. A planner that invests time in a description of the free space so that many later problems in the same space are solved quickly is a **multi-query planner** (G-2786); a planner that solves each problem from nothing, as fast as it can, is a **single-query planner** (G-2787) (MR §10.1; Hauser ch.10, "Multi-query PRMs"). MR §10.5 classes the PRM as a multi-query planner and the RRT as a single-query one.

When the room changes. The investment pays only while the room stays the same. If the furniture moves, every milestone and every edge must be checked again, because each was checked against the old room (MR §10.1; Kavraki et al. 1996). For a room that changes often, a single-query planner that grows only what one trip needs is the better choice (MR §10.1). The four trips are now answered; but every one of them was found only because the roadmap happened to cover the room well, which the next section examines.

4. What makes a graph a roadmap: accessibility and connectivity

> **Key point:** A graph serves every trip only if two things hold. Accessibility: from any free configuration, a simple free path reaches the graph. Connectivity: two configurations joined in free space are joined in the graph, so the graph has one piece for each piece of free space. With 10 milestones our roadmap breaks the second rule and trips 2 and 4 fail; from 20 milestones on it is one piece and every trip is found.

The four trips of Section 3 all succeeded with 100 milestones. Would fewer do? Figure 4 rebuilds the roadmap from the first 5, 10, 20, 50 and 100 milestones, the kept random configurations, of the same run. Watch the colours, one per piece of the roadmap, and the red squares, free floor that cannot reach any of its 10 nearest milestones by a free straight segment.

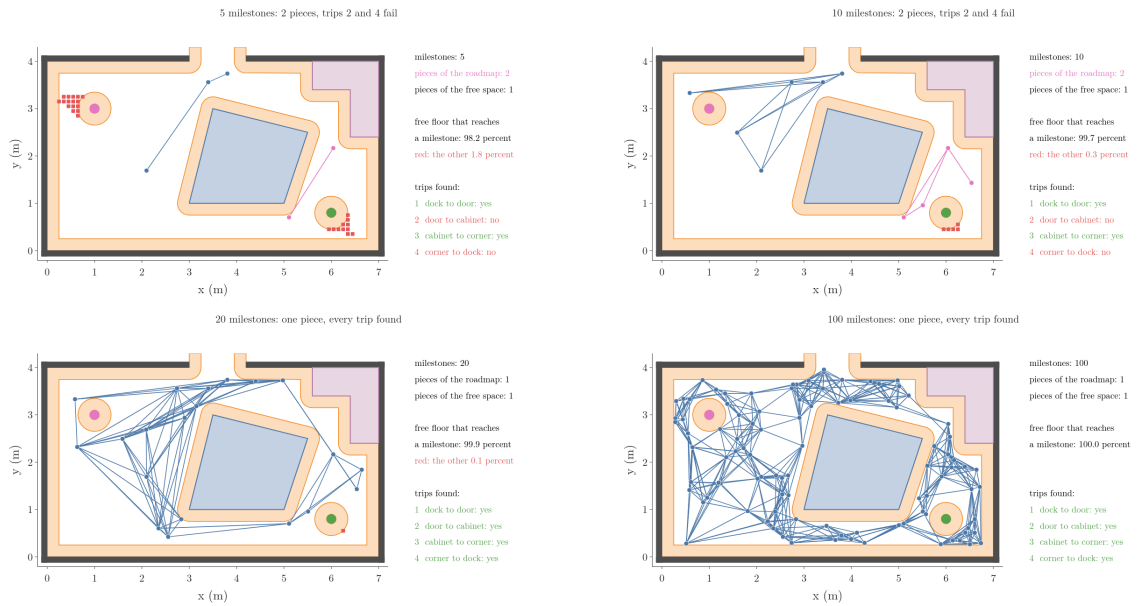


Figure 4: Animation: the roadmap from the first 5, 10, 20, 50 and 100 milestones of seed 0, each joined to its 10 nearest; each piece of the roadmap has its own colour; red squares mark free floor (tested every 0.1 m) with no free straight segment to one of its 10 nearest milestones. 5 milestones: 2 pieces, 98.2 percent of the free floor reaches a milestone, trips 2 and 4 fail. 10 milestones: 2 pieces (6 and 4 milestones), 99.7 percent, trips 2 and 4 fail. 20 milestones: one piece, 99.9 percent, every trip found. 100 milestones: one piece, 100.0 percent

Two different things went wrong with few milestones:

- **Some free floor cannot reach the graph.** With 5 milestones, 1.8 percent of the free floor, behind the pink pole and behind the green pole, sees none of them along a free straight segment. A trip starting there could not even join the roadmap.
- **The graph is in two pieces where the free space is one.** The room's free space is one piece (connected components of free space), but with 10 milestones the roadmap has 2 pieces, 6 milestones round the left and top, 4 on the right. Trip 2 (door to cabinet) and trip 4 (corner to dock) join different pieces, and A* finds no route, though the robot could drive both.

The two failures break the two conditions a graph must meet to be a **roadmap** (G-2783), a graph in free space that can answer every query (LaValle §6.1; MR §10.3):

1. **Accessibility** (G-2788): from any free configuration, a free path to some point of the roadmap is easy to compute, such as a straight segment to a nearby milestone.
2. **Connectivity** (G-2789), which LaValle §6.1 calls connectivity-preserving: if a free path joins two configurations, the roadmap joins their entry points too; equivalently, there is one piece of the roadmap for every connected component (G-2656), each piece, of the free space (MR §10.3).

With both, every trip is three short steps: from the start onto the roadmap, along the roadmap, and off to the goal (MR §10.3). In Figure 4, from 20 milestones on, the roadmap is

one piece and 99.9 percent or more of the free floor reaches it, and all four trips are found.

A probabilistic roadmap is called probabilistic because random milestones meet the two conditions only with some probability: as more milestones are added, the chance that the roadmap joins every start and goal that the free space joins goes to 1 (Hauser ch.10, "Analysis of visibility and performance"). With 20 milestones, seed 0 met them; another seed might not. How likely a wrong "no route" is, is the subject of the next section.

5. Can a planner miss a route? Completeness

> **Key point:** A complete planner always finds a route if one exists and says so if none does; a resolution-complete planner does this down to the size of its grid; a probabilistically complete planner, such as the PRM, finds an existing route with a chance that rises to 1 as it samples more. A route the PRM returns is always free; its "no route" can be wrong.

5.1 Complete and resolution complete

Section 4 ended with the roadmap of 10 milestones answering "no route" for trip 2, which has one. So "no route" from a planner may be false, and the planners we know differ in how false it can be. Three grades are used (MR §10.1; KAIST PRM slides):

1. A planner is **complete** (G-2790) if it always finds a route when one exists and reports failure when none exists (MR §10.1).
2. A planner is **resolution complete** (G-2791) if it does so at the resolution of its grid: it finds every route that exists through the grid, and may miss a route that needs a finer grid (MR §10.1).
3. A planner is **probabilistically complete** (G-2792) if the chance that it finds an existing route goes to 1 as its planning time, here its number of samples, grows (MR §10.1).

Complete. A complete planner needs an exact description of the free space, such as the exact room in C-space built for the disk robot, or an exact roadmap of it. For a robot moving among polygons in the plane, joining the obstacle corners that see each other gives such an exact roadmap, and A* on it is complete (MR §10.3). For most robots, building that description is too costly, so resolution or probabilistic completeness is the best a practical planner gets (MR §10.3).

Resolution complete: a grid misses the corner. Figure 5 plans trip 3 on two grids of configurations: the robot's centre at the middle of each cell, a point kept if the robot is free there, neighbours in 8 directions joined by free segments. With 0.5 m cells, 44 of the 112 points are free, none of them near the corner behind the green pole, and the grid answers "no route" to the corner. With 0.25 m cells, 244 of 448 points are free and the grid finds a route of 1.903 m (notebook).

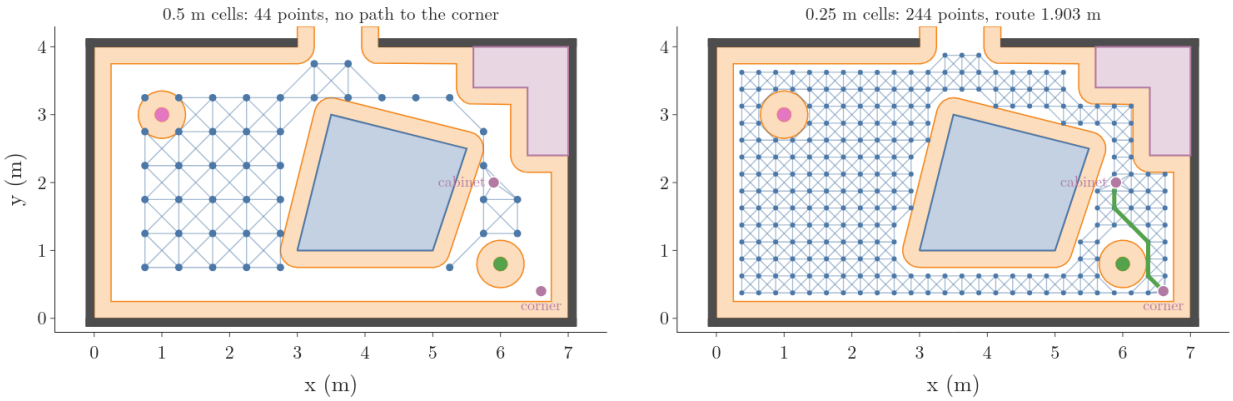


Figure 5: Two grids of configurations in the room's C-space (obstacles grown by 0.25 m, orange), each point the robot's centre at the middle of a cell, kept if the robot is free there, with free segments to its neighbours in 8 directions (light blue). Left, 0.5 m cells: 44 points, none near the corner (6.6, 0.4), no route from the cabinet to the corner. Right, 0.25 m cells: 244 points and the route cabinet to corner, 1.903 m (green)

The robot fits beside and under the green pole, with 0.4 m and 0.2 m to spare, but the gaps fall between the 0.5 m grid's points. So the grid's "no route" is true only at its resolution. (The grid graph of the room had 76 free cells of 112 because it counted cells with no furniture in them; here a point is free only if the whole 0.25 m disk is.) The PRM has no grid; its only knob is how many samples it draws.

5.2 Probabilistically complete

Figure 6 runs the PRM 200 times for each number of milestones (the random free configurations it keeps), from 5 to 100, and counts how often each trip is found. Every trip has a route, so each curve is the chance of a correct answer; watch each curve climb towards 100 percent.

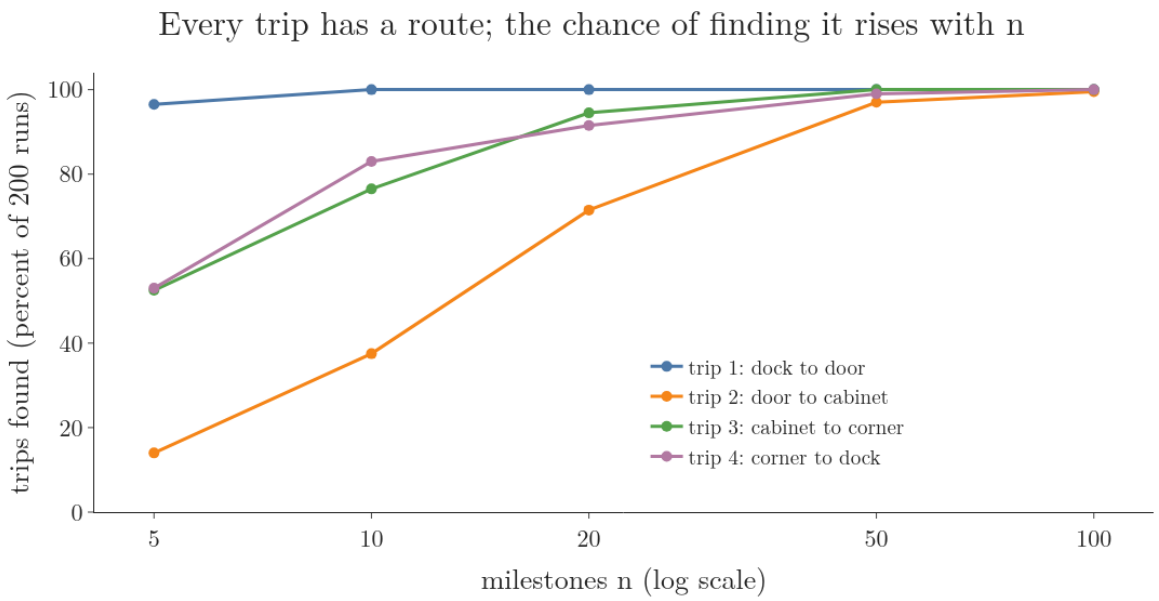


Figure 6: Chart: the percentage of 200 runs in which a roadmap of n uniform milestones (10 nearest neighbours) finds each trip, for n = 5, 10, 20, 50 and 100 (log scale). Trip 1 dock to door: 96.5 percent at n = 5, 100 from n = 10. Trip 2 door to cabinet: 14, 37.5, 71.5, 97 and 99.5 percent. Trip 3 cabinet to corner: 52.5, 76.5, 94.5, 100, 100. Trip 4 corner to dock: 53, 83, 91.5, 99, 100

The PRM is probabilistically complete (Hauser ch.10; MR §10.5):

- **A route it returns is always free,** because every edge of it passed the collision check (Hauser ch.10; KAIST PRM slides), as far as that check can tell: edge resolution tests points 0.05 m apart, so a collision thinner than that could slip through (step size and order).
- **Its "no route" can be wrong,** as trip 2 shows: with 10 milestones it is found in only 37.5 percent of runs. The chance of that wrong answer falls towards 0 as the number of milestones grows: trip 2 is found in 99.5 percent of runs with 100 milestones (notebook).

Trip 2 lags because the cabinet spot sits in a pocket between the table, the cabinet and the green pole, which few straight lines reach: it sees 17.0 percent of the free floor, against 27.9 percent for the corner and 57.9 percent for the door (notebook). The next part turns that share into a chance.

5.3 Why more milestones help: the chance of missing

A trip end joins the roadmap only if some milestone sees it along a free straight segment. Figure 7 shades the free floor that sees the corner: 27.9 percent of it (notebook). Every milestone is drawn uniformly from the free floor, so each one lands in the shaded part with the probability (G-1574) 0.279, about 0.28.

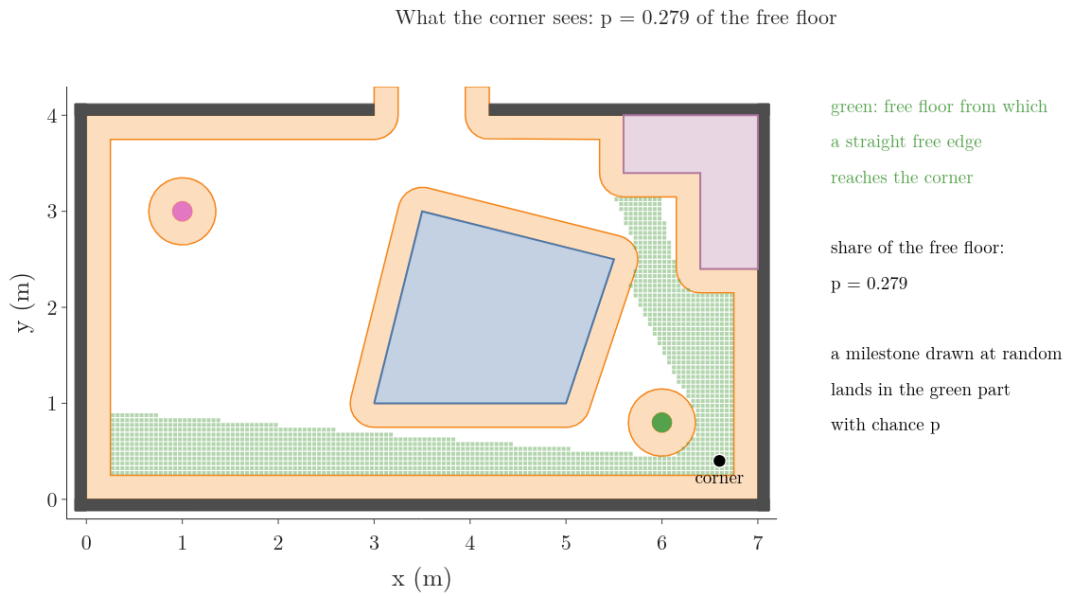


Figure 7: The free floor from which a straight free segment reaches the corner (6.6, 0.4), shaded green, tested every 0.05 m: the column between the cabinet's grown base and the right wall, and the strip under the green pole and along the bottom wall; it is 0.279 of the free floor (p = 0.279). A milestone drawn at random from the free floor lands in the green part with chance p

So one milestone misses the shaded part with probability:

$$1 - 0.28 = 0.72$$

The milestones are drawn independently (G-934): one landing says nothing about another. So the chance that two both miss is the product (G-1576) of their chances:

$$0.72 \times 0.72 = 0.518$$

and that five all miss:

$$0.72^5 = 0.193$$

So with 5 milestones some milestone sees the corner with chance:

$$1 - 0.193 = 0.807$$

In 200 runs with 5 milestones, the corner saw a milestone in 82.0 percent (notebook). With 10 milestones, the same steps give:

$$0.72^{10} = 0.0374$$

$$1 - 0.0374 = 0.963$$

and the 200 runs gave 97.0 percent. The same steps in symbols, with:

- p is the share of the free space that sees a configuration q along a free straight segment, 0.28 for the corner;
- n is the number of milestones, each drawn uniformly and independently from the free space, 5 above;
- P_{miss} is the probability that no milestone sees q.

$$P_{\text{miss}} = (1-p)(1-p)\cdots(1-p)$$

$$= (1-p)^n$$

(Hauser ch.10, "Analysis of visibility and performance"). For any p above 0, multiplying by a number below 1 again and again drives P_{miss} to 0, which is why more samples make a miss ever less likely.

The catch: small p. How fast P_{miss} falls depends on p. For a configuration that sees 1 percent of the free space:

$$0.99^{100} = 0.366$$

$$0.99^{300} = 0.049$$

Even 300 milestones leave a 4.9 percent chance of missing it: on average about 1 divided by p milestones are needed before one sees it, 100 here (Hauser ch.10). The guarantee also needs a route with some room to spare: a gap exactly as wide as the robot leaves a route of zero width, which no random sample ever lands on (Hauser ch.10, "given some relatively mild assumptions on the shape of the free space"; KAIST PRM slides). Small shares of free space that a route must cross are where uniform sampling struggles, which is the tray problem of the opening.

6. Narrow passages and smarter sampling

> **Key point:** A narrow passage is a thin part of free space that a route must cross; uniform samples rarely land in it, so the roadmap splits on either side. Gaussian sampling keeps samples next to obstacles, and the bridge test keeps the midpoint of two colliding samples, which lands in thin gaps; with 50 milestones they find the tray robot's trip in

86 and 93 percent of runs, against 32 percent for uniform sampling.

6.1 The tray robot: where uniform samples starve

The opening's second problem: the robot carries a round tray 0.8 m across. The tray is now the widest part of the robot, so its footprint is a disk of radius 0.4 m, and every obstacle grows by 0.4 m instead of 0.25 m (growing the obstacles). The room is the same; only this radius changes in this section. We widen the robot instead of changing the room because the 0.25 m robot meets no gap narrow enough to starve uniform samples: with 50 milestones, its trip from the dock to the cabinet is found in 93 percent of runs (notebook). The gaps the robot must use to reach the cabinet from its dock shrink (notebook):

- under the table, the 1.0 m between the table and the bottom wall leaves the centre a corridor 2 m long and this wide:

$$1.0 - 2 \times 0.4 = 0.2 \text{ m}$$

- beside the table, its right corner (5.5, 2.5) is 0.9 m from the cabinet's lower part, and both grow by 0.4 m:

$$0.9 - 2 \times 0.4 = 0.1 \text{ m}$$

- beside the table, the green pole's centre is 1.012 m from the table's right edge; the table grows by 0.4 m and the pole, 0.1 m in radius, by 0.4 m to 0.5 m:

$$1.012 - 0.4 - 0.5 = 0.112 \text{ m}$$

A thin part of free space that a route has to cross, such as these gaps, is a **narrow passage** (G-2793) (Hauser ch.10; LaValle §5.6.1, "Some analysis"). The corridor is 0.4 m^2 of the 9.96 m^2 of free floor (notebook):

$$0.2 \times 2.0 = 0.4 \text{ m}^2$$

$$0.4/9.96 = 0.040$$

So 50 milestones, the roadmap's random free configurations, drawn uniformly put on average 2 into the corridor:

$$50 \times 0.040 = 2.0$$

Two milestones cannot form a chain along 2 m of corridor 0.2 m wide. In 100 runs with 50 milestones, the dock-to-cabinet trip is found in 32 percent for the tray robot, against 93 percent for the robot without the tray (notebook). The top panel of Figure 8 shows a typical

failure: the roadmap splits into a piece of 45 milestones round the dock and a piece of 5 beside the cabinet.

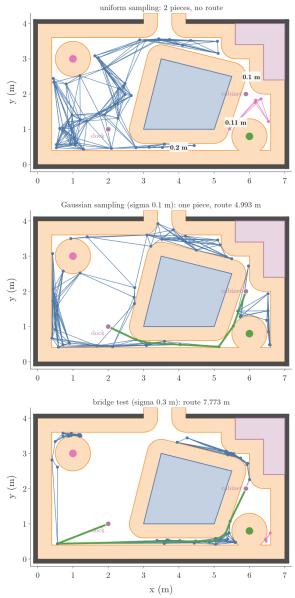


Figure 8: The C-space of the robot carrying a tray 0.8 m across (obstacles grown by 0.4 m, orange), with the gaps marked: 0.2 m under the table, 0.1 m between the table and the cabinet, 0.11 m between the table and the green pole. One run (seed 5001) of 50 milestones by each strategy, 10 nearest neighbours, trip dock to cabinet; each colour is one piece of the roadmap. Top, uniform sampling: 2 pieces, 45 milestones (blue) and 5 beside the cabinet (pink), no route. Middle, Gaussian sampling, standard deviation 0.1 m: one piece, the milestones lie along the obstacles, route 4.993 m (green) through the corridor under the table. Bottom, bridge test, standard deviation 0.3 m: the milestones sit in the thin gaps, route 7.773 m

More uniform samples help slowly: with 100 milestones the trip is found in 66 percent of runs, with 200 in 93 percent (notebook). The trouble is where the samples go, not how many there are, so the fix is to draw them where the passages are.

6.2 Gaussian sampling and the bridge test

Narrow passages always lie next to obstacles, and the thinnest lie between two obstacles close together (Hauser ch.10; LaValle §5.6.3). Two ways of drawing samples use these facts, each from a pair of draws (Hauser ch.10; LaValle §5.6.3; WPI L11):

1. **Gaussian sampling** (G-2794): draw q_1 uniformly, then q_2 near it, its offset in x and in y drawn from a normal distribution (G-1343) with mean 0 and standard deviation (G-1871) 0.1 m. Keep the free one if exactly one of the two is free; otherwise drop both.
2. **The bridge test** (G-2795): draw q_1 uniformly; go on only if it collides. Draw q_2 near it, with standard deviation 0.3 m; go on only if it collides too. Keep the midpoint m of the two if m is free.

Why they land in the right places. A Gaussian pair with one end free and one in collision straddles the edge of an obstacle, so the kept end lies within about 0.1 m of an obstacle (Hauser ch.10). A bridge has both ends inside obstacles and a free middle, so it spans a thin gap between two obstacles, like a plank across a stream: a wide open area rarely has obstacles on both sides within a short reach (LaValle §5.6.3; WPI L11).

Figure 9 runs both, draw by draw. In the first part, Gaussian sampling, watch draw 3: q_1 at (0.36, 2.221) lies in the grown left wall, q_2 at (0.407, 2.343) is free, so q_2 is kept. In the second part, the bridge test, watch draw 4. With:

- q_1 is the first draw, (3.9359, 0.6002), just inside the grown table, whose edge is at y = 0.6;
- q_2 is the second draw, (3.8372, 0.3626), inside the grown bottom wall, whose edge is at y = 0.4;
- m is their midpoint, each coordinate the average of the two.

$$m_x = (3.9359 + 3.8372)/2 = 3.8866$$

$$m_y = (0.6002 + 0.3626)/2 = 0.4814$$

The midpoint lies in the corridor under the table, between y = 0.4 and y = 0.6, so it is free and kept.

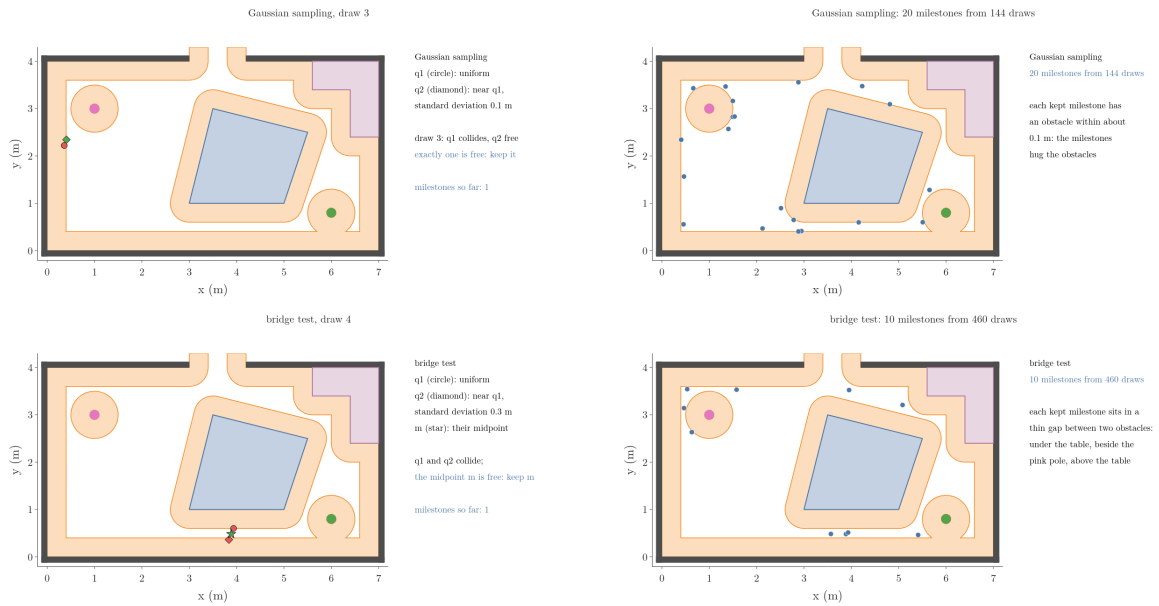


Figure 9: Animation in the tray robot's C-space (obstacles grown by 0.4 m, orange); green: free, red: in collision, blue: kept as a milestone. Part 1, Gaussian sampling (standard deviation 0.1 m): q1 (circle) uniform and q2 (diamond) near it, one draw per frame; draw 3, q1 in the grown left wall and q2 free, keeps q2. Last frame of part 1: 20 milestones from 144 draws, all close to obstacles. Part 2, the bridge test (standard deviation 0.3 m): q1, then q2 if q1 collides, then their midpoint m (star) if both collide; draw 4 has both ends in collision, one in the grown table and one in the grown bottom wall, and a free midpoint in the corridor, which is kept. Last frame: 10 milestones from 460 draws, in thin gaps under the table, beside the pink pole and above the table

The bridge test throws most draws away: 10 milestones took 460 draws in Figure 9. A draw costs only a configuration check or three, far less than an edge, so the trade pays where narrow passages decide the answer (Hauser ch.10). Over 100 runs of the tray robot's trip with 50 milestones (notebook):

Sampling	Trip found	Draws	Configurations checked
uniform	32 percent	142	4,529
Gaussian, 0.1 m	86 percent	536	5,311
bridge test, 0.3 m	93 percent	2,470	8,881
uniform, 200 milestones	93 percent	566	10,660

The bridge test reaches 93 percent with 8,881 checks; uniform sampling needs 200 milestones and 10,660 checks for the same 93 percent. Figure 8 shows one run, seed 5001, chosen because it has the commonest outcome of the 100: uniform sampling fails while both others succeed (53 of the 100 runs; notebook). Its bottom panel also shows the bridge test's weakness: its milestones crowd into gaps and leave the open floor empty, so its route of 7.773 m wanders round the room.

The tray robot now gets through its gaps. Every milestone so far, though, has stayed in the roadmap, needed or not; the next section keeps only the ones that add something.

7. The visibility roadmap: guards and connectors

> **Key point:** The visibility roadmap keeps a sample only if it sees no kept guard (then it becomes a guard) or it sees guards of two pieces of the roadmap (then it becomes a connector joining them). In our room, 5 guards and 4 connectors, 9 vertices and 8 edges, cover the free floor that took 100 milestones and 525 edges before.

The roadmap of Section 3 has 100 milestones (kept random configurations) and 525 checked edges, and most of them repeat each other: a milestone in the middle of the open floor sees the same floor as its neighbours. A smaller roadmap is quicker to search and to store, which matters when it is built once and queried many times (LaValle §5.6.2).

The **visibility roadmap** (G-2796) keeps a free sample only if it adds something, by one of two rules, and discards the rest (LaValle §5.6.2; KAIST PRM slides):

1. a sample that sees no **guard** (G-2797), no kept vertex of the first kind, along a free straight segment becomes a guard: it adds floor that no guard sees yet;
2. a sample that sees guards from two different pieces of the roadmap becomes a **connector** (G-2798), joined by edges to one guard of each piece: it joins the pieces;
3. any other free sample sees guards of one piece only, adds nothing, and is discarded.

Every sample is tested against every guard, however far away, because a guard's view is everything it sees (LaValle §5.6.2). Figure 10 runs the rules on 300 draws, seed 0. Watch each new guard shade the floor it sees, the connectors (green diamonds) join the pieces, and the grey dots, discarded samples, fill the floor already seen.

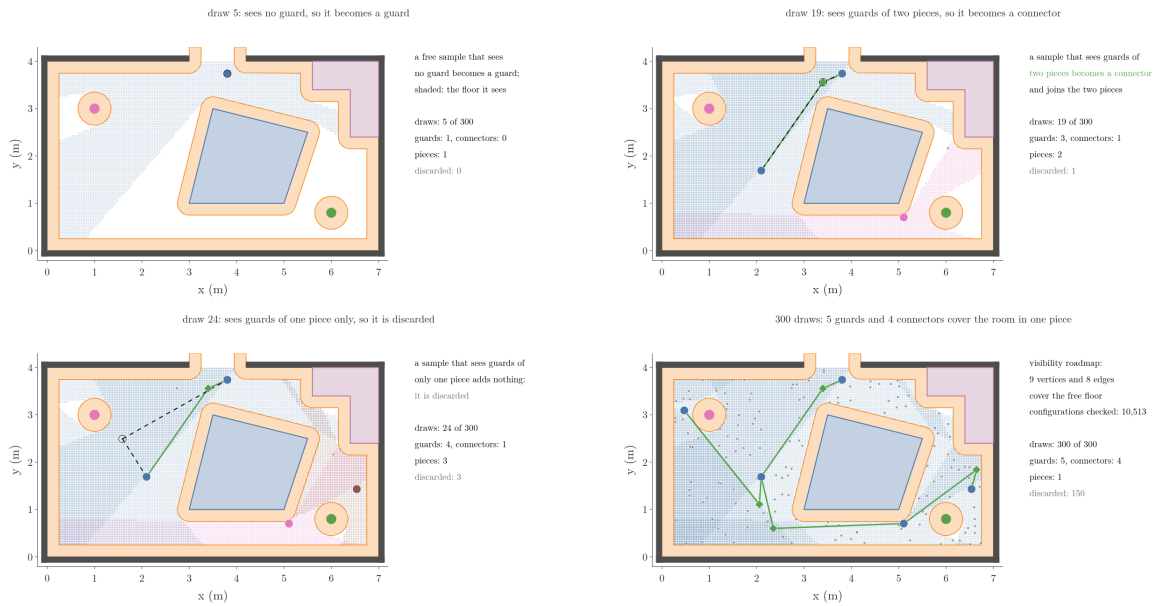


Figure 10: Animation: the visibility roadmap in the room's C-space, 300 uniform draws (seed 0). A free sample that sees no guard becomes a guard (big dot), and the floor it sees is shaded in its piece's colour (draws 5, 8, 10, 20 and 238). A sample that sees guards of two pieces becomes a connector (green diamond) with edges to one guard of each (draws 19, 28, 40 and 240), and the pieces merge. Other free samples are discarded (grey dots, 150 of them), such as draw 24, which sees guards of one piece only. Last frame: 5 guards, 4 connectors, 8 edges, one piece; 10,513 configurations checked

Of the 300 draws, 141 collided, 150 were discarded, and 9 were kept: 5 guards and 4 connectors, joined by 8 edges into one piece (notebook). The last new vertex came at draw 240; the 60 draws after it found nothing new, a sign that the guards see the whole room. The four trips, each end joined to the vertices it sees, give routes of 3.4, 9.047, 1.888 and 4.784 m.

The small roadmap has two costs (LaValle §5.6.2; KAIST PRM slides):

- **Building it costs more per sample:** each sample is tested against every guard, so 300 draws took 10,513 configuration checks, against 7,934 for the 100-milestone roadmap.
- **Its routes can be long:** trip 2 is 9.047 m here against 3.587 m on the 100-milestone roadmap, because only 9 vertices are there to route through.

Narrow passages stay as hard for it as for the basic roadmap, since a guard or connector must still land there (KAIST PRM slides). One name clash: some books call the graph of obstacle corners that see each other a visibility roadmap too (MR §10.3, Exercise 10.7); the visibility roadmap of this section is the guard-and-connector graph of LaValle §5.6.2. Both roadmaps have spent most of their checks on edges, which the last section avoids.

8. Lazy collision checking

> **Key point:** Most edges of a roadmap are never used by any route, yet each costs dozens of configuration checks. Lazy collision checking joins milestones without checking the edges, searches, and checks only the edges of the route found, deleting any that collide and searching again: trip 2 needs 11 edge checks, 129 configuration checks, against 7,954 when every edge is checked first.

The visibility roadmap saved milestones; the edges still cost the most. Building the roadmap of Section 2 took 7,745 configuration checks for its edges, against 189 for its milestones (its random free configurations), yet trip 2's route uses 3 of its 525 edges. Checking an edge is the main cost of a roadmap, especially for detailed robots, where every configuration check tests many bodies and pairs of links (Hauser ch.10, "Lazy collision checking in PRMs").

Lazy collision checking (G-2799) delays an edge's check until a route wants to use it (Hauser ch.10):

1. join every milestone, the start and the goal to their 10 nearest, assuming every edge is free;
2. search for a route with A*; if there is none, report failure;
3. check the route's edges, in order from the start;
4. if an edge collides, delete it from the graph and go back to step 2;
5. if every edge is free, return the route.

The bet is that an edge between two free milestones is usually free too (Hauser ch.10). Figure 11 runs trip 2 on the 100 milestones of Section 2. Watch the orange candidate routes: an edge that passes the check turns green; an edge that cuts the grown table turns red and is deleted, and A* searches again.

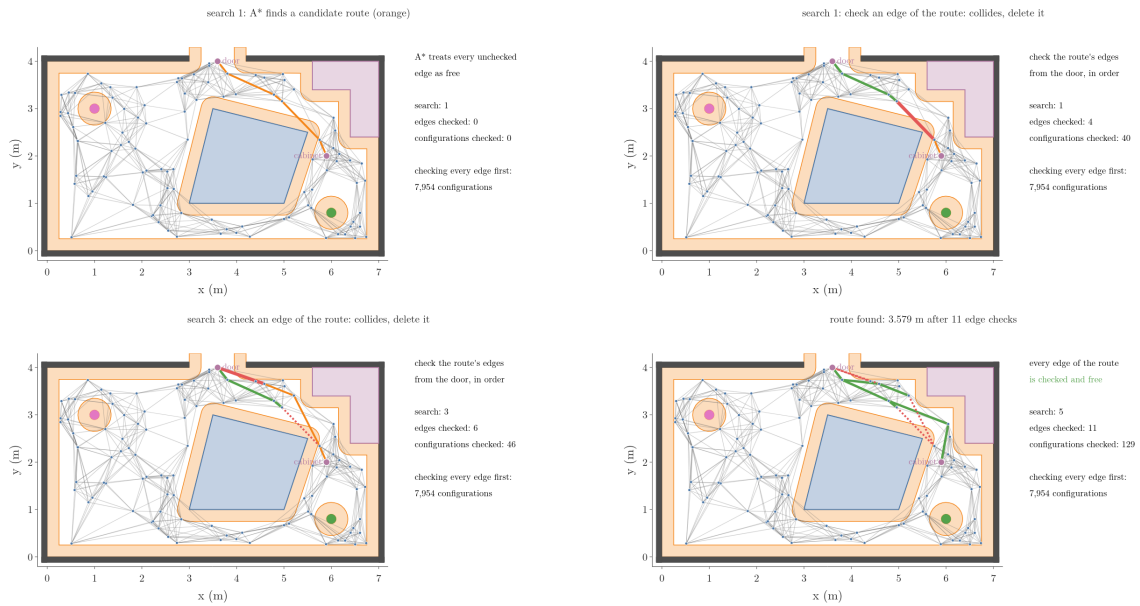


Figure 11: Animation: lazy collision checking on trip 2, door (3.6, 4.0) to cabinet (5.9, 2.0), with the 100 milestones of seed 0. Grey: edges to the 10 nearest, not checked. Search 1: A* finds a candidate route (orange); its edges are checked from the door, three are free (green), the fourth cuts the grown table (red) and is deleted. Searches 2 and 3 each find a colliding edge at once; search 4 checks two free edges and finds the fourth collision; search 5 finds a route whose edges are all free. Last frame: route 3.579 m after 5 searches and 11 edge checks, 129 configurations checked, against 7,954 when every edge is checked first

Lazy checking needed 5 searches and 11 edge checks, of which 4 collided; it checked 129 configurations and found a route of 3.579 m (notebook). Checking every edge first, as in Section 2, costs 7,745 checks for the edges plus 209 for trip 2's ends:

$$7{,}745 + 209 = 7{,}954$$

$$129/7{,}954 = 0.016$$

So the lazy route cost 1.6 percent of the checks. The price is extra searches, cheap in a graph of 100 milestones (Hauser ch.10).

9. Summary

Idea	What it does	On our room
Probabilistic roadmap	random free milestones joined to their 10 nearest by checked edges	100 milestones, 525 edges, 7,934 checks
Query phase	join start and goal, then A*	trip 1: 3.531 m for 226 checks
Multi-query planner	builds once, answers many trips	8,776 checks for four trips, against 32,578
Accessibility and connectivity	every configuration reaches the graph; one piece per piece of free space	10 milestones: 2 pieces, trips 2 and 4 fail
Resolution complete	finds routes down to the grid's size	0.5 m grid: no route to the corner; 0.25 m: 1.903 m
Probabilistically complete	misses an existing route with chance $(1-p)^n$	corner: 0.193 with 5 milestones
Narrow passages	thin gaps that uniform samples miss	tray robot: found in 32 percent of runs
Gaussian sampling, bridge test	samples next to obstacles, or in thin gaps	86 and 93 percent with 50 milestones
Visibility roadmap	keeps only guards and connectors	9 vertices and 8 edges
Lazy collision checking	checks only the edges a route uses	trip 2: 129 checks, against 7,954

In the table, p is the share of the free space that sees a configuration, 0.28 for the corner, and n the number of milestones.

- The robot builds one graph of its free space from random samples and answers all four trips from it: 7,934 checks for the build and 842 for the trips, against 32,578 for a new graph per trip, because a roadmap holds no start or goal and a trip only joins its two ends; with the tray, Gaussian sampling and the bridge test get it through the 0.1 m to 0.2 m gaps in 86 and 93 percent of runs, against 32 percent with uniform samples, because they put milestones where the gaps are.
- A **probabilistic roadmap** (G-2780) never builds the obstacle region, so it scales to robots with many joints, where a grid of 10 values per joint needs 10,000,000 cells for 7 joints.
- The **learning phase** (G-2784) spends most of its checks on edges, 7,745 against 189 for the milestones, because each edge is checked at many configurations; the **query phase** (G-2785) costs at most 2.8 percent of the build.
- A **multi-query planner** (G-2786) pays off only in an unchanged room, because every edge was checked against the room as it was; a **single-query planner** (G-2787) suits rooms that change often.
- A graph answers every trip only if it meets **accessibility** (G-2788) and **connectivity**

(G-2789), so a roadmap of 10 milestones in 2 pieces misses trips 2 and 4 in a free space of one piece.

- The PRM is **probabilistically complete** (G-2792): a returned route is always free, but "no route" can be wrong, with a chance $(1-p)^n$ that falls to 0, slowly when p is small; a grid is only **resolution complete** (G-2791), and its "no route" holds only at its cell size.
- A **narrow passage** (G-2793) has a small share of the free space, so uniform samples rarely land in it: the tray robot's 0.2 m corridor takes 4 percent of the free floor, and its trip is found in 32 percent of runs.
- **Gaussian sampling** (G-2794) and the **bridge test** (G-2795) put milestones next to obstacles or in thin gaps, so the tray robot's trip is found in 86 and 93 percent of runs with the same 50 milestones.
- A **visibility roadmap** (G-2796) keeps only **guards** (G-2797) and **connectors** (G-2798), so 9 vertices cover the room, at more checks per sample and with longer routes.
- **Lazy collision checking** (G-2799) checks only the edges of candidate routes, because most edges are never used: 129 configuration checks for trip 2 instead of 7,954.

The robot can now plan many trips in an unchanged room from one roadmap, cross narrow gaps and check only what it uses. Still missing: every trip so far moves the robot alone, past obstacles that never move; picking up the cup and carrying it changes the free space as the cup moves, which is manipulation planning.

10. Sources

Built from

- Aaron Becker (University of Houston), "PRM: Probabilistic Roadmap Method in 3D and with 7-DOF robot arm", YouTube, https://www.youtube.com/watch?v=tlFVbHENPCI
- Northwestern Robotics, "Modern Robotics, Chapter 10.1: Overview of Motion Planning", YouTube, 2:52–3:58, https://www.youtube.com/watch?v=aC4LQuB4Cic (MR 10.1)
- Northwestern Robotics, "Modern Robotics, Chapter 10.3: Complete Path Planners", YouTube, https://www.youtube.com/watch?v=SIHzI0jsLK0 (MR 10.3)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §10.1 "Overview of Motion Planning" (multiple-query and single-query planning, completeness), §10.3 "Complete Path Planners" (roadmap properties, visibility graph), §10.5 "Sampling Methods for Motion Planning", §10.5.2 "The PRM Algorithm" (Algorithm 10.4). Free preprint: http://modernrobotics.org (MR §10.1, §10.3, §10.5, §10.5.2)
- Hauser, K. *Robotic Systems*, University of Illinois. Ch.10 "Motion Planning in Higher Dimensions": "Probabilistic roadmaps", "Analysis of visibility and performance", "Nonuniform sampling strategies", "Multi-query PRMs", "Lazy collision checking in PRMs", https://motion.cs.illinois.edu/RoboticSystems/MotionPlanningHigherDimensions.html (Hauser ch.10)
- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §5.6.1 "The Basic Method" (preprocessing and query phases, "Some analy-

sis"), https://lavalle.pl/planning/node240.html; §5.6.2 "Visibility Roadmap", https://lavalle.pl/planning/node245.html; §5.6.3 "Heuristics for Improving Roadmaps" (Gaussian sampling, bridge-test sampling), https://lavalle.pl/planning/node246.html; §6.1 "Roadmaps" (accessibility, connectivity-preserving), https://lavalle.pl/planning/node259.html (LaValle §5.6.1, §5.6.2, §5.6.3, §6.1)

- KAIST CS686 (Sung-Eui Yoon), "Probabilistic Roadmaps" lecture slides: completeness, multi-query PRM, the answer's reliability, visibility-based PRM, Gaussian sampling, https://sgvr.kaist.ac.kr/~sungeui/MPA_F19/Slides/Lecture5.pdf (KAIST PRM slides)
- Li, J. (WPI), RBE 550 Motion Planning, Lecture 11 "Sampling-Based Planning" slides: the bridge sampler, https://users.wpi.edu/~zli11/teaching/rbe550_2018/LectureSlide_PDF/L11-SamplingBasedPlanning_02.pdf (WPI L11)

Other references

- Kavraki, L. E., Švestka, P., Latombe, J.-C. and Overmars, M. H. (1996). "Probabilistic roadmaps for path planning in high-dimensional configuration spaces". *IEEE Transactions on Robotics and Automation* 12(4), 566–580; abstract (learning phase, query phase, local planner, static workspaces) checked in its technical report UU-CS-1994-32, https://dspace.library.uu.nl/handle/1874/17328 (Kavraki et al. 1996)

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Milestone (G-2781)	A free configuration kept as a vertex of a probabilistic roadmap, drawn at random; milestones are the places the roadmap's routes pass through.
Local planner (G-2782)	The fast method a sampling planner uses to try to join two configurations, usually the straight segment; it proposes one simple motion without searching round obstacles, and a collision check reports whether that motion is free.
Probabilistic roadmap (PRM) (G-2780)	A graph of random free configurations (milestones) joined by straight edges that pass a collision check, built once and then searched with A*; it describes the free space without building the obstacle region, so it scales to robots with many joints.

Term	Meaning
Learning phase (preprocessing phase) (G-2784)	The first phase of a roadmap planner: sample milestones and connect them into the roadmap, done once for a space so that later queries are quick.
Query phase (G-2785)	The second phase of a roadmap planner: join a start and a goal to the roadmap by the local planner and search the graph for a route; it costs far less than building the roadmap.
Multi-query planner (G-2786)	A planner that invests time in a reusable description of the free space, such as a roadmap, so that many later problems in the same unchanged space are solved quickly.
Single-query planner (G-2787)	A planner that solves each planning problem from nothing, as fast as it can, such as the RRT; it suits spaces that change often, where a stored description would go stale.
Roadmap (G-2783)	A graph in free space that can answer every query: any free configuration reaches it easily (accessibility) and it has one piece for each piece of free space (connectivity), so a route is start to roadmap, along it, and off to the goal.
Accessibility (roadmap) (G-2788)	The roadmap condition that from any free configuration a free path to the roadmap, such as a straight segment to a nearby milestone, is easy to compute, so every trip can join the roadmap.
Connectivity (connectivity-preserving roadmap) (G-2789)	The roadmap condition that configurations joined in free space are joined in the roadmap too, one roadmap piece per piece of free space, so no existing route is missed.
Complete planner (G-2790)	A planner that always finds a route when one exists and reports failure when none exists; it needs an exact description of the free space, so it is practical only for simple problems.
Resolution complete (G-2791)	Complete down to the resolution of a grid: the planner finds every route that exists through its grid, but may answer no route where only a finer grid would find one.

Term	Meaning
Probabilistically complete (G-2792)	Finding an existing route with a chance that goes to 1 as the planner samples more; a returned route is always free, but an answer of no route can be wrong.
Narrow passage (G-2793)	A thin part of free space that a route must cross, such as a gap only slightly wider than the robot; uniform random samples rarely land in it, so roadmaps tend to split on either side.
Gaussian sampling (G-2794)	Drawing a uniform configuration and a second one near it (normal offsets) and keeping the free one only if exactly one of the two is free, so milestones gather next to obstacles, where narrow passages lie.
Bridge test (G-2795)	Keeping the midpoint of two nearby configurations that both collide, if the midpoint is free, so milestones land in thin gaps between obstacles, where narrow passages are.
Visibility roadmap (G-2796)	A probabilistic roadmap that keeps a sample only if it sees no guard (it becomes a guard) or sees guards of two pieces (it becomes a connector), giving a small roadmap that covers the free space.
Guard (visibility roadmap) (G-2797)	A vertex of a visibility roadmap that sees no other guard along a free straight segment; guards are added to cover free space no guard sees yet.
Connector (visibility roadmap) (G-2798)	A vertex of a visibility roadmap that sees guards of two different pieces of the roadmap and is joined to both, so it merges the pieces.
Lazy collision checking (G-2799)	Delaying an edge's collision check until a candidate route uses it, deleting edges found to collide and searching again; it saves the checks of the many edges no route uses.
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).

Term	Meaning
Rapidly-exploring random tree (RRT) (G-2761)	A sampling-based planner that grows a tree from the start: each round it draws a random configuration, steps the tree's nearest node at most a fixed distance towards it and keeps the new node if the step is free, until the tree reaches the goal; it explores fast but its routes zig-zag.
Obstacle region (C-obstacle) (G-2650)	The set of all configurations at which the robot's body touches or overlaps an obstacle, C_{obs}; drawing it in C-space lets a planner test a placement as one point, such as the disk robot's centre (3.5, 3.2), which lies in it 0.2 m from the table.
Grown obstacle (inflated obstacle) (G-2652)	An obstacle with every point within the robot's radius added, its edges pushed out and its corners rounded; for a disk robot it is exactly the obstacle region, so the robot can be treated as a point (the table, 3.625 m^2, grows to 5.747 m^2 for a 0.25 m radius).
Curse of dimensionality (G-520)	The problems that appear when data has too many dimensions: lower performance and more computation.
Collision detection (G-2665)	Deciding, yes or no, whether two bodies overlap, such as the robot's disk and the table; planners call it thousands of times instead of building the obstacle region.
Sampling-based planner (G-2760)	A motion planner that never builds the obstacle region or a grid, but draws random configurations and tests them with collision checks, joining the free ones into a tree or graph; it works in C-spaces with many dimensions, where grids are far too large.
Continuous uniform distribution $U(a, b)$ (G-467)	A continuous variable spread evenly between a and b, with density $1/(b-a)$.
Nearest neighbours (neighbours) (G-1306)	The observations at the smallest distance from a given point; in KNN, the k training points closest to the query point, whose values or labels decide the prediction.
Edge resolution (segment checking) (G-2679)	Checking a straight path segment in C-space for collisions by testing points along it, spaced at most a chosen step apart; too long a step can miss a thin collision, too short a step wastes time.

Term	Meaning
Steer (steering function) (G-2763)	The step of a sampling-based planner that moves from a tree node towards a sample, at most a fixed distance (0.5 m here), stopping on the sample if it is closer; it keeps every new edge short enough to check.
Graph (G-2540)	A set of vertices (points) joined by edges (lines), such as the room's 76 free cells joined by 114 moves; it keeps only which places connect directly, so one search method works for maps, puzzles and any problem of states and moves.
Heuristic h (G-2570)	A cheap guess of the cost still to go from a state to the goal, such as 3.121 m from the start cell to the door; a search uses it to prefer states that look nearer the goal.
Connected component (of free space) (G-2656)	A piece of free space within which the robot can move from any configuration to any other without touching an obstacle; a path between two configurations exists only if both lie in the same component.
Probability (G-1574)	A number from 0 to 1 measuring how likely an event is; written $P(A)$.
Independent events (G-934)	Events where one happening does not change the probability of the other; for them the probability that both happen is the product of their probabilities.
Product rule for independent events (G-1576)	For independent events, the probability that both happen is the product of their separate probabilities: $P(A \cap B) = P(A) \times P(B)$.
Normal distribution (G-1343)	A symmetric, bell-shaped distribution: most values sit near the mean and they get rarer the further out we go; its mean and standard deviation set its centre and width.
Standard deviation (G-1871)	How far values typically lie from the mean; divide by $n - 1$ for a sample (pandas default) or by n for a population (NumPy default).